

Atividade Prática: Random Forest

Renan Costa Alencar

Maio 2026

1 Missão Prática: A Sabedoria da Floresta

1.1 Contexto do Problema

Na missão anterior, lutámos contra o Overfitting tentando "podar" uma única Árvore de Decisão. Aprendemos que uma árvore isolada possui alta variância: ela decora os dados de treino e é muito sensível a ruídos.

Nesta missão, em vez de confiarmos o diagnóstico do hospital a um único médico, vamos criar um "conselho médico" utilizando o algoritmo Random Forest. O objetivo é provar empiricamente que a combinação de múltiplas árvores reduz o Gap entre treino e teste.

1.2 A Tarefa

Utilizando o Google Colab e o mesmo dataset de Risco de Diabetes (Wisconsin Breast Cancer), deverá implementar as três configurações abaixo e observar o poder computacional do modelo RandomForestClassifier.

1. A Árvore Solitária (Baseline): Instancie uma Árvore de Decisão sem limites para relembrar o problema.
Dica: `DecisionTreeClassifier(random_state=42)`
2. A Pequena Floresta: Instancie uma Random Forest com apenas 10 árvores.
Dica: `RandomForestClassifier(n_estimators=10, random_state=42)`
3. A Grande Floresta Paralelizada: Instancie uma Random Forest com 150 árvores. Utilize o parâmetro `n_jobs=-1` para dizer ao Python para usar todos os núcleos do processador e treinar as árvores em paralelo.
Dica: `RandomForestClassifier(n_estimators=150, n_jobs=-1, random_state=42)`
4. A Floresta Podada (O Estado da Arte): Instancie uma Random Forest com 150 árvores, mas agora limite a profundidade máxima de cada árvore a 5 níveis. Este é o modelo que junta o poder da multidão com a sabedoria da poda.
Dica: `RandomForestClassifier(n_estimators=150, max_depth=5, n_jobs=-1, random_state=42)`

Importante: Random State e Dados Utilize sempre `random_state=42` nos modelos e mantenha a divisão de dados (`test_size=0.3`) do código base. Isto garante o rigor científico da experiência: para comparar algoritmos de forma justa, o conjunto de dados de avaliação tem de ser mantido estritamente constante.

1.3 Entregáveis (Relatório)

Preencha a tabela comparativa abaixo. Em seguida, responda às perguntas de análise arquitetural com a sua perspectiva de engenharia.

1.3.1 1. Tabela Comparativa de Métricas (Resultados Obtidos)

Modelo Configurado	Accuracy (Treino)	Accuracy (Teste)	Gap
Árvore Solitária	100.00%	91.81%	8.19%
Pequena Floresta	100.00%	92.98%	7.02%
Grande Floresta	100.00%	94.15%	5.85%
Floresta Podada	99.25%	94.15%	5.09%

Table 1: Resultados Obtidos

1.3.2 2. Perguntas de Análise (A Decisão do Engenheiro)

A. Ao comparar o Modelo 3 (Floresta livre) com o Modelo 4 (Floresta Podada), é provável que repare num detalhe crucial: o Modelo 3 pode apresentar uma acurácia de Teste ligeiramente superior, mas o Modelo 4 possui um Gap (diferença entre Treino e Teste) significativamente menor.

Como Engenheiro de Dados num sistema crítico de um hospital, qual dos dois modelos recomendaria colocar em produção? Justifique a sua escolha abordando o compromisso entre “Pico de Acurácia” e “Previsibilidade/Estabilidade” (o clássico Trade-off Viés-Variância).

Resposta: Como Engenheiro de Machine Learning em um sistema de saúde de alta criticidade (como o diagnóstico de câncer de mama), recomendo com total convicção colocar em produção o **Modelo 4 (Floresta Podada)**.

Justificativa Técnica baseada no Trade-off Viés-Variância:

- 1. Previsibilidade e Estabilidade sobre Picos Marginais:** Embora o Modelo 3 (Floresta Livre) possa apresentar uma acurácia de teste sutilmente superior em cenários de validação isolados, ele exibe um gap de generalização significativamente maior. O Modelo 3 possui viés zero no treino (100% de acurácia), decorando o ruído dos dados de treino (alta variância). O Modelo 4, ao aplicar a poda (`max_depth=5`), limita a profundidade das árvores, adicionando um viés controlado, mas reduzindo drasticamente a variância. Isso garante que o desempenho do modelo no mundo real seja consistente e previsível.

2. **Mitigação de Riscos de Saúde:** Em diagnósticos médicos, a consistência do modelo sob dados ruidosos, novos equipamentos ou diferentes distribuições de pacientes é essencial. Um modelo com menor gap de generalização (Modelo 4) reduz a probabilidade de comportamentos erráticos ou falhas críticas invisíveis (como falsos negativos) sob novos dados.
3. **Eficiência Computacional:** Árvores podadas são consideravelmente menores, ocupando menos memória RAM no servidor e garantindo uma latência de inferência ultrabaixa.

B. O parâmetro `n_jobs=-1` distribui o treino das 150 árvores por todos os núcleos da CPU. Qual é a característica fundamental da arquitetura da Random Forest que permite esta paralelização impecável (ao contrário de algoritmos onde o passo N depende do passo $(N - 1)$)?

Resposta: A paralelização impecável da Random Forest (escalabilidade perfeita com `n_jobs=-1`) é viabilizada por sua arquitetura baseada no princípio do **Bagging (Bootstrap Aggregating)**.

Características Fundamentais:

1. **Independência Completa das Árvores:** Ao contrário de algoritmos de Boosting (como Gradient Boosting, XGBoost, LightGBM), onde cada árvore N é construída de forma estritamente sequencial para corrigir os erros cometidos pelas árvores 0 a $N - 1$, na Random Forest **todas as árvores são completamente independentes**.
2. **Amostragem via Bootstrap:** Cada árvore é ajustada em um subconjunto de dados de treino amostrado com reposição (bootstrap) de forma isolada.
3. **Subespaço de Recursos Aleatórios:** A seleção de variáveis (features) para divisão em cada nó ocorre aleatoriamente para cada árvore individual. Devido a essa ausência total de dependência temporal ou sequencial entre as árvores, o problema de treinamento é classificado como *embarrassingly parallel* (embaraçosamente paralelo). O framework divide o trabalho entre todos os núcleos de CPU disponíveis de forma simultânea, sem custos de sincronização durante a etapa de treinamento.